

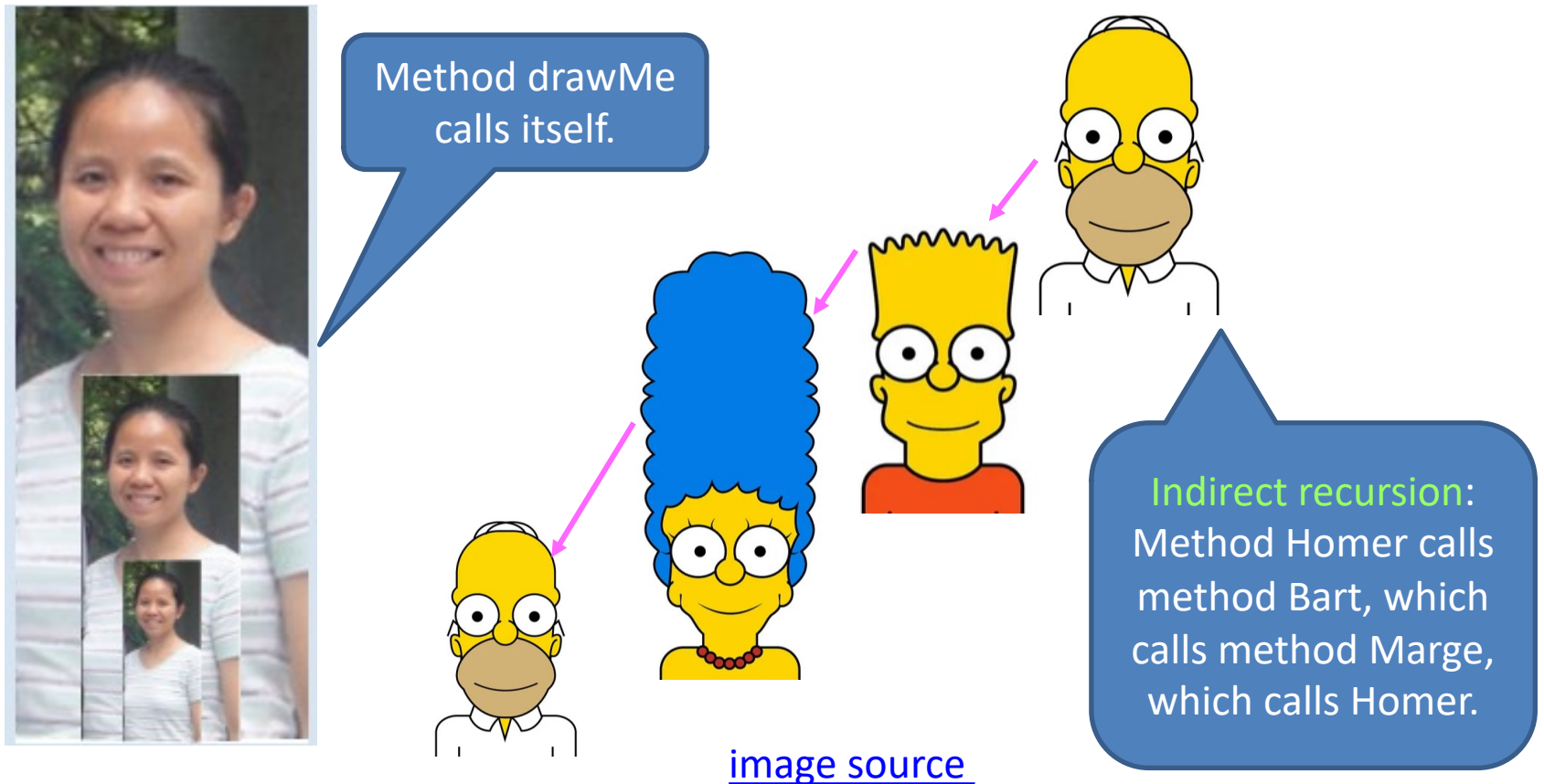
Introduction to Recursion

Goals

- What is **recursion**?
- Why **recursion**?
- Examples of **recursion**
 - Calculate factorial
 - Print a string in reverse order
 - Find out the sum of an array of integers

What is recursion?

- A method calls itself directly or indirectly



Motivation

WHY RECURSION

Why recursion?

- new mode of thinking
- powerful programming tool
- divide-and-conquer paradigm
 - divide
 - conquer
 - combine

Solve problems using recursion

- **Divide** the problem into pieces and **ASSUME** all problems with **smaller** sizes can be solved.
- **Combine** answers to **smaller** size problems to obtain answer to **the original** problem.
- **Explicitly Conquer** (assign a direct answer to) problem with **smallest** size (base case).

Key ideas in Recursion

- Each recursive call must **reduce** the problem size.
- A base case provides a **direct** answer.
- Recursion must **stop** — it cannot continue indefinitely.



Recursion Template

- **To solve a problem recursively:**
- **Base case**
 - Solve the smallest problem directly
- **Recursive case**
 - Reduce the problem to a smaller version
 - Call the function on the smaller problem
- **Combine**
 - Use the result of the smaller problem to solve the original

Example

FACTORIAL

Factorial: non-recursive definition

- Factorial $n!$, when n is positive, is the products of n and all the positive integers below it, ie,

$$n! = n * (n-1) * (n-2) * \dots * 1$$

- Define $0! = 1$.
- What can $n!$ represent?
 - Number of all possible ways to line up n objects.
 - For the above five suspects, there are a total of $5! = 120$ ways to line them up.



Factorial: recursive definition

- By definition

$$n! = n * \underline{(n-1)} * \underline{(n-2)} * \dots * \underline{1}$$

and

$$(n-1)! = \underline{(n-1)} * \underline{(n-2)} * \dots * \underline{1}$$

- Represent $n!$ in terms of $(n-1)!$

$$n! = n * (n-1)!$$

Components of recursive definition

- A recursive definition is made up of two parts.
 - a **base case** tells us directly what the answer is.

$$0! = 1$$

- a **recursive case** defines the answer in terms of the answer to some other related problem.

- When $n \geq 1$, construct $n!$ on the basis of $(n-1)!$

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n * (n - 1)! & \text{otherwise} \end{cases}$$

Calculate factorial n! using Recursion

//return type is long since factorial
//grows very fast. Assume that $n \geq 0$.

```
long factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    else {  
        return n * factorial(n-1);  
    }  
}
```

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n * (n-1)! & \text{otherwise} \end{cases}$$

Illustration: calculate factorial(3)

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n * (n - 1)! & \text{otherwise} \end{cases}$$

$$\text{factorial}(3) = 3 * \text{factorial}(2)$$

$$\text{factorial}(2) = 2 * \text{factorial}(1)$$

$$\text{factorial}(1) = 1 * \text{factorial}(0)$$

$$\text{factorial}(0) = 1$$

Illustration: calculate factorial(3)

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n * (n - 1)! & \text{otherwise} \end{cases}$$

$$\text{factorial}(3) = 3 * \text{factorial}(2) = 3 * 2 = 6$$

$$\text{factorial}(2) = 2 * \text{factorial}(1) = 2 * 1 = 2$$

$$\text{factorial}(1) = 1 * \text{factorial}(0) = 1 * 1 = 1$$

$$\text{factorial}(0) = 1$$

Implement function calls

- When a caller function calls a callee,
 - Caller function saves its local variables.
 - Pass parameters to callee function.
 - Switch control from caller to callee.
 - Run codes in callee functions.
 - Get return if applicable.
 - Switch control back from callee to caller, all local variables in callee will be released.
 - Restore local variables of caller.

Function calls illustration

- What if Function A calls Function B, which in turn calls Function C?
- What happens to the local variables of Functions A and B when Function C is called?
- Recursive calls are also function calls.
 - A recursive function just calls itself with **smaller** size.

Illustrate Recursive Calls

- Each recursive call is stored on the call stack
- The system unwinds the stack after reaching the base case.

Print a String in Backward Order

- For example, given “abc de”, print “de cba” (without quotes).
- What is the **shortest string**? How to print it?
- For a string longer than a shortest string, how to **divide** it into smaller cases?
- **Combine** the print of shorter strings to get the print of the original string.

Summary

- In recursive function, find out **base case**.
- **Assume** that all **smaller case** can be solved, **combine** those smaller solutions to get a solution to the original problem.
- When you call a function itself, the size must be reduced, or system needs to allocate infinite spaces to hold the setting of previous running instances, and the system will crash.